SMA N 12 BANDUNG

# MODUL AJAR

# BERPIKIR KOMPUTASIONAL ALGORITMA PEMROGRAMAN

Arjun Yuda Firwanda, S.Tr.Kom

## A. IDENTITAS MODUL

| Komponen | Keterangan |
|---|---|
| Satuan Pendidikan | SMA Negeri 12 Bandung |
| Mata Pelajaran | Informatika |
| Fase / Kelas | Fase F / Kelas XII |
| Kelas Sasaran | XII-3 IPA (dapat disesuaikan untuk kelas XII lain) |
| Semester | Ganjil |
| Tahun Pelajaran | 2026/2027 |
| Nama Guru | Arjun Yuda Firwanda, S.Tr.Kom. |
| Materi Pokok | Berpikir Komputasional dan Algoritma Pemrograman |
| Submateri | Berpikir komputasional; struktur data sebagai pengantar; algoritma; deskriptif, pseudocode, flowchart; simbol flowchart; dasar bahasa C/C++ Arduino; sintaks, variabel, konstanta, serial, pin digital/analog, percabangan, dan perulangan. |
| Alokasi Waktu | 5 JP = 225 menit |
| Rincian Pertemuan | Pertemuan 1: 3 JP (135 menit). Pertemuan 2: 2 JP (90 menit). |
| Model Pembelajaran | Inquiry Learning |
| Pendekatan | Kontekstual, eksploratif, berpusat pada peserta didik, diskusi dan latihan terbimbing. |
| Produk/hasil belajar | Penjelasan lisan, rancangan algoritma dalam notasi yang sesuai, dan latihan membaca/menyusun potongan kode sederhana. |
| Kedudukan LKPD | LKPD digunakan untuk latihan. LKPD tidak dipakai sebagai lembar tes atau alat asesmen sumatif. |

## B. RASIONAL

Berpikir komputasional merupakan cara berpikir sistematis untuk memahami dan menyelesaikan persoalan secara efisien. Kemampuan ini tidak terbatas pada kegiatan menggunakan komputer. Peserta didik dapat menerapkannya ketika mengatur kegiatan kelas, menyusun urutan kerja, mengenali pola dalam data, memilah informasi yang penting, dan menentukan langkah penyelesaian masalah.

Setelah mengenali cara berpikir komputasional, peserta didik mempelajari algoritma sebagai urutan langkah yang terstruktur. Algoritma dapat ditulis menggunakan bahasa

sehari-hari secara deskriptif, pseudocode, atau flowchart. Ketiga representasi tersebut membantu peserta didik memeriksa apakah langkah penyelesaian sudah runtut, masuk akal, dan dapat dipahami oleh orang lain.

Materi kemudian dihubungkan dengan pemrograman C/C++ pada Arduino. Peserta didik mengenali struktur dasar sketch, sintaks, komentar, variabel, konstanta, komunikasi serial, fungsi pin I/O, percabangan, dan perulangan. Penguasaan konsep-konsep tersebut membantu peserta didik melihat hubungan antara rancangan algoritma dengan instruksi yang dapat dijalankan oleh perangkat komputasi.

Inquiry Learning dipilih karena materi ini cocok dipelajari melalui pertanyaan, dugaan awal, pencarian informasi dari salindia dan contoh, pengujian gagasan, serta penarikan kesimpulan. Guru bertindak sebagai fasilitator yang menyiapkan masalah dan pertanyaan penuntun. Peserta didik diberi ruang untuk menyampaikan alasan, membandingkan representasi, dan memperbaiki pemahaman setelah memperoleh bukti atau umpan balik.

Pemetaan cakupan materi:

| Kelompok materi | Isi yang tampak pada salindia | Penempatan |
|---|---|---|
| Pengantar dan berpikir komputasional | Tujuan pembelajaran; pengantar struktur data; pengertian berpikir komputasional; dekomposisi, abstraksi, algoritma, dan pengenalan pola. | Pertemuan 1 |
| Algoritma dan representasi | Pengertian algoritma; notasi deskriptif, pseudocode, dan flowchart; simbol flowchart. | Pertemuan 1 |
| Dasar C/C++ Arduino | Struktur dasar pemrograman, block comment, semicolon, curly brace, #define, #include, variabel, dan konstanta. | Pertemuan 2 |
| Komunikasi dan pin I/O | Serial.begin(), Serial.print(), Serial.println(), pinMode(), digitalRead(), digitalWrite(), analogRead(), analogWrite(). | Pertemuan 2 |
| Kontrol alur program | Percabangan if dan if-else; perulangan for, while, dan do-while. | Pertemuan 2 |

## C. KARAKTERISTIK PESERTA DIDIK

Gambaran kebutuhan belajar:

Peserta didik kelas XII dapat memiliki tingkat pengalaman yang beragam dalam menyusun algoritma dan membaca kode. Sebagian peserta didik mungkin sudah familiar dengan urutan instruksi atau pemrograman, sedangkan peserta didik lain masih membutuhkan contoh konkret sebelum memahami istilah abstrak seperti abstraksi, variabel, kondisi, dan perulangan. Modul ini memfasilitasi kedua kondisi tersebut melalui pertanyaan kontekstual, representasi visual, contoh bertahap, diskusi, dan latihan dengan tingkat dukungan yang berbeda.

Pengetahuan bersyarat:

1) Peserta didik dapat memahami instruksi sederhana dalam bahasa sehari-hari dan menyampaikan urutannya secara lisan.
2) Peserta didik mengenal fungsi dasar perangkat komputer atau gawai serta contoh penggunaan teknologi dalam kehidupan sehari-hari.
3) Peserta didik dapat membaca angka, membandingkan kondisi, dan mengikuti urutan langkah.
4) Pengalaman awal pemrograman tidak diasumsikan sama; guru melakukan pemetaan singkat melalui tanya jawab.

Potensi hambatan:

| Hambatan yang mungkin muncul | Dukungan yang disiapkan guru |
|---|---|
| Peserta didik tertukar antara dekomposisi, abstraksi, pengenalan pola, dan algoritma. | Gunakan satu masalah kontekstual yang sama, lalu tunjukkan peran masing-masing konsep pada masalah tersebut. |
| Peserta didik memahami deskripsi algoritma tetapi kesulitan mengubahnya menjadi pseudocode atau flowchart. | Berikan contoh langkah kecil, bank kata/struktur pseudocode, dan template simbol flowchart. |
| Peserta didik belum mengenali sintaks Arduino seperti titik koma atau kurung kurawal. | Tampilkan kode pendek, beri penanda visual pada bagian-bagian penting, dan baca kode baris demi baris. |
| Peserta didik menyamakan fungsi input, output, dan pembacaan analog. | Gunakan tabel perbandingan fungsi, contoh tombol/LED/sensor, serta pertanyaan “data masuk atau keluar?” |

| Hambatan yang mungkin muncul | Dukungan yang disiapkan guru |
|---|---|
| Peserta didik hanya menghafal if dan loop tanpa memahami alurnya. | Minta peserta didik memprediksi keluaran kode terlebih dahulu, kemudian membuktikannya secara bertahap. |
| Peserta didik pasif saat diskusi. | Gunakan waktu berpikir mandiri, diskusi berpasangan, lalu kesempatan berbicara secara bergiliran. |

## D. CAPAIAN PEMBELAJARAN

Setelah mengikuti rangkaian pembelajaran, peserta didik diharapkan mampu menggunakan cara berpikir komputasional untuk menganalisis persoalan; menjelaskan algoritma dan membandingkan bentuk representasinya; mengenali simbol flowchart; serta memahami fungsi unsur dasar pemrograman C/C++ Arduino, termasuk variabel, konstanta, komunikasi serial, fungsi pin I/O, percabangan, dan perulangan. Peserta didik menunjukkan proses berpikir melalui penjelasan, alasan, dan latihan terbimbing.

## E. TUJUAN PEMBELAJARAN

| Pertemuan | Tujuan Pembelajaran | Indikator ketercapaian |
|---|---|---|
| 1 (3 JP / 135 menit) | Peserta didik mampu menjelaskan pengertian berpikir komputasional dan menguraikan empat konsep utamanya. | Menjelaskan makna dekomposisi, abstraksi, algoritma, dan pengenalan pola serta memberikan contoh yang relevan. |
| 1 (3 JP / 135 menit) | Peserta didik mampu menjelaskan pengertian algoritma dan membandingkan tiga bentuk penulisannya. | Membedakan notasi deskriptif, pseudocode, dan flowchart berdasarkan bentuk, kelebihan, dan keterbatasannya. |
| 1 (3 JP / 135 menit) | Peserta didik mampu mengenali fungsi simbol-simbol flowchart yang disajikan dalam materi. | Mengaitkan terminator, process, decision, flow, input/output, stored data, connector, manual operation, delay, document, alternate process, pre-defined process, display, manual input, dan preparation dengan fungsinya. |
| 2 (2 JP / 90 menit) | Peserta didik mampu menjelaskan unsur dasar sintaks Arduino dan membedakan variabel serta konstanta. | Menjelaskan komentar, semicolon, curly brace, #define, #include, aturan penamaan variabel, TRUE/FALSE, HIGH/LOW, INPUT/OUTPUT. |

| Pertemuan | Tujuan Pembelajaran | Indikator ketercapaian |
|---|---|---|
| 2 (2 JP / 90 menit) | Peserta didik mampu menguraikan fungsi komunikasi serial dan fungsi pin I/O yang dicantumkan pada materi. | Membedakan Serial.begin(), Serial.print(), Serial.println(), pinMode(), digitalRead(), digitalWrite(), analogRead(), dan analogWrite(). |
| 2 (2 JP / 90 menit) | Peserta didik mampu menjelaskan kontrol alur percabangan dan perulangan serta menalar alur kode sederhana. | Membedakan if, if-else, for, while, dan do-while serta memberikan alasan pemilihan struktur yang sesuai. |

Indikator sikap dan proses:

1) Peserta didik mengajukan pertanyaan yang berkaitan dengan masalah dan konsep yang sedang dipelajari.
2) Peserta didik menyampaikan dugaan awal serta alasan yang mendukung dugaan tersebut.
3) Peserta didik menggunakan informasi dari salindia, contoh guru, dan diskusi untuk memeriksa pemahamannya.
4) Peserta didik bersedia memperbaiki jawaban setelah menerima pertanyaan penuntun atau bukti tambahan.
5) Peserta didik menghargai pendapat teman dan menjelaskan pemikirannya dengan bahasa yang dapat dipahami.

## F. PEMAHAMAN BERMAKNA

1) Persoalan yang tampak rumit dapat diselesaikan dengan lebih teratur ketika dipecah menjadi bagian-bagian kecil.
2) Informasi yang relevan perlu dipilih agar proses berpikir tidak terjebak pada detail yang tidak membantu penyelesaian masalah.
3) Pola yang berulang dapat membantu seseorang mengenali struktur masalah dan mengembangkan solusi yang dapat digunakan kembali.
4) Algoritma menjelaskan langkah penyelesaian dengan urutan yang dapat diperiksa dan dikomunikasikan.

5) Notasi deskriptif, pseudocode, dan flowchart dapat digunakan untuk menyampaikan algoritma; tiap bentuk memiliki kelebihan dan keterbatasan.
6) Program merupakan instruksi terstruktur. Kesalahan kecil pada sintaks atau logika dapat mengubah cara program berjalan.
7) Percabangan menentukan tindakan berdasarkan kondisi, sedangkan perulangan membantu menjalankan instruksi secara berulang.

## G. PERTANYAAN PEMANTIK

Pertemuan 1:

1) Mengapa kita perlu membagi pekerjaan besar menjadi beberapa pekerjaan yang lebih kecil?
2) Ketika menyelesaikan masalah, apakah semua informasi harus diperhatikan? Mengapa?
3) Bagaimana kita mengetahui bahwa dua masalah memiliki pola penyelesaian yang mirip?
4) Apa perbedaan antara menjelaskan langkah secara lisan dengan menggambarkannya menggunakan flowchart?
5) Mengapa algoritma perlu ditulis dengan langkah yang jelas dan berurutan?
6) Jika satu simbol flowchart digunakan secara keliru, apakah orang lain masih mungkin memahami alurnya? Jelaskan.

Pertemuan 2:

1) Mengapa komputer atau papan Arduino membutuhkan instruksi yang ditulis dengan aturan tertentu?
2) Apa yang dapat terjadi jika titik koma atau kurung kurawal ditempatkan secara keliru?
3) Bagaimana sebuah program dapat menerima informasi dan memberikan respons?
4) Bagaimana program memilih tindakan ketika kondisi tertentu terpenuhi atau tidak terpenuhi?
5) Kapan sebuah instruksi perlu diulang? Apakah semua perulangan berlangsung dengan cara yang sama?

6) Bagaimana kita dapat memprediksi perilaku program sebelum mencoba menjalankannya?

## H. SARANA, PRASARANA, DAN SUMBER BELAJAR

| Komponen | Kebutuhan dan alternatif |
|---|---|
| Sumber utama | Presentasi "Berpikir Komputasional dan Algoritma Pemograman" yang dilampirkan (57 slide). |
| Perangkat utama | Komputer/laptop guru, proyektor atau layar, papan tulis, spidol, dan alat tulis peserta didik. |
| Perangkat pendukung | Komputer peserta didik, Arduino IDE atau simulator Arduino apabila tersedia. Kegiatan tetap dapat dilakukan dengan analisis contoh kode tercetak/tampil di layar bila perangkat tidak tersedia. |
| Media visual | Ilustrasi algoritma, contoh pseudocode/flowchart, tabel fungsi, dan potongan kode yang ditampilkan guru. |
| Bahan latihan | LKPD latihan di bagian lampiran. LKPD digunakan untuk eksplorasi dan penguatan, bukan sebagai lembar tes/asesmen. |
| Pengaturan kelas | Tempat duduk menghadap layar untuk pemodelan konsep; diskusi berpasangan atau kelompok kecil saat menyelesaikan latihan. |
| Kesiapan guru | Memeriksa urutan salindia, menyiapkan contoh kontekstual, memastikan kode pada layar terbaca, serta menandai koreksi teknis yang perlu dijelaskan dengan hati-hati. |

## I. DESAIN PEMBELAJARAN INQUIRY LEARNING

Inquiry Learning menempatkan peserta didik sebagai pihak yang aktif menyelidiki pertanyaan. Dalam modul ini, inquiry tidak berarti peserta didik dibiarkan mencari tanpa arahan. Guru memberi masalah kontekstual, mengajukan pertanyaan penuntun, menyediakan sumber informasi, dan membantu peserta didik memeriksa alasan yang digunakan untuk menarik kesimpulan.

Sintaks Inquiry Learning:

| Tahap | Peran guru | Aktivitas peserta didik | Bukti proses belajar |
|---|---|---|---|
| 1. Orientasi / stimulasi | Menyajikan masalah dekat dengan kehidupan peserta didik dan menghubungkannya dengan materi. | Mengamati, mengingat pengalaman, dan menyebutkan apa yang sudah diketahui. | Respons awal dan keterlibatan dalam diskusi. |
| 2. Merumuskan masalah/pertanyaan | Membimbing peserta didik merumuskan hal yang perlu dijelaskan atau dibuktikan. | Menyusun pertanyaan yang jelas tentang konsep, urutan langkah, simbol, atau perilaku kode. | Pertanyaan lisan atau catatan pertanyaan. |
| 3. Merumuskan dugaan/hipotesis | Meminta peserta didik memprediksi jawaban beserta alasan awal. | Menyampaikan dugaan sebelum melihat penjelasan atau membandingkan jawaban. | Dugaan dan alasan yang disampaikan. |
| 4. Mengumpulkan data/informasi | Mengarahkan peserta didik membaca salindia, mengamati contoh, dan mengumpulkan informasi yang relevan. | Mencatat informasi penting, membandingkan contoh, dan mendiskusikan makna istilah. | Catatan ringkas, jawaban pada latihan, atau penjelasan lisan. |
| 5. Menguji dugaan | Mengajukan pertanyaan pembanding, meminta peserta didik menelusuri langkah atau kode, dan memberi umpan balik. | Memeriksa apakah dugaan sesuai dengan definisi, contoh, atau alur program; memperbaiki jawaban jika diperlukan. | Alasan, revisi pemahaman, dan penjelasan hasil penelusuran. |
| 6. Merumuskan kesimpulan | Membantu merangkum prinsip penting dan mengaitkannya dengan kehidupan nyata. | Menyampaikan kesimpulan dengan bahasa sendiri dan menuliskan hal yang masih perlu dipelajari. | Kesimpulan lisan dan refleksi akhir. |

Prinsip fasilitasi guru:

1) Pertanyaan guru diarahkan untuk menggali alasan, bukan hanya meminta definisi.
2) Jawaban awal yang belum tepat digunakan sebagai bahan klarifikasi, bukan untuk mempermalukan peserta didik.
3) Guru memberi waktu berpikir sebelum meminta jawaban; diskusi berpasangan dapat digunakan agar lebih banyak peserta didik terlibat.
4) Contoh kode diperiksa secara konseptual langkah demi langkah. Jika tidak ada perangkat untuk menjalankan program, peserta didik melakukan penelusuran logika secara manual.
5) LKPD latihan tidak dijadikan tes. Guru menggunakan percakapan, observasi proses, dan refleksi sebagai sumber informasi tentang perkembangan pemahaman.

## J. MATERI PEMBELAJARAN

1. Pengantar Struktur Data

   Pada salindia pengantar, struktur data dijelaskan sebagai cara menyimpan dan mengatur data agar mudah digunakan. Contoh sederhana yang ditampilkan ialah daftar belanja dan daftar hadir. Dalam informatika, bentuk yang disebutkan meliputi list, array, dan tabel.

   Hubungan dengan berpikir komputasional: ketika sebuah persoalan melibatkan banyak informasi, informasi tersebut perlu diatur agar mudah dibaca, dibandingkan, dan diproses. Peserta didik tidak dituntut mempelajari implementasi teknis struktur data secara mendalam dalam dua pertemuan ini; istilah tersebut berfungsi sebagai pengantar tentang pentingnya mengelola informasi secara teratur.

   Contoh pertanyaan guru: “Jika kita ingin mengetahui peserta didik yang hadir, apakah nama-nama lebih mudah diperiksa bila ditulis acak atau disusun dalam daftar? Informasi apa yang perlu ada dalam daftar hadir?”

2. Berpikir Komputasional

Berpikir komputasional adalah cara berpikir untuk menyelesaikan persoalan secara sistematis dan efisien, yang dapat dibantu atau diimplementasikan oleh komputer. Inti penekanannya terletak pada cara menyusun masalah dan solusi dengan jelas; komputer dapat membantu menjalankan langkah tersebut, tetapi proses berpikir juga dapat dilakukan manusia tanpa komputer.

Konsep utama:

| Konsep | Makna sederhana | Contoh kontekstual | Pertanyaan penuntun |
|---|---|---|---|
| Dekomposisi | Memecah masalah besar menjadi bagian yang lebih kecil. | Mempersiapkan presentasi kelas: menentukan topik, membagi bagian, menyiapkan media, dan berlatih. | Bagian apa saja yang membentuk masalah ini? |
| Abstraksi | Memusatkan perhatian pada informasi penting dan mengabaikan detail yang tidak diperlukan untuk menyelesaikan masalah. | Saat mencari ruang kelas, informasi nomor ruang dan lokasi lebih relevan daripada warna sepatu orang yang lewat. | Informasi mana yang benar-benar membantu? |
| Algoritma | Menyusun langkah penyelesaian secara terurut dan jelas. | Langkah meminjam buku: mencari buku, mencatat identitas, menyerahkan buku untuk diproses, lalu menerima informasi peminjaman. | Apa langkah pertama, berikutnya, dan terakhir? |
| Pengenalan pola | Menemukan kemiripan atau pola berulang pada masalah/contoh. | Melihat pola jadwal piket yang berulang setiap minggu. | Bagian apa yang sama pada beberapa kejadian? |

Keempat konsep dapat saling mendukung. Peserta didik dapat memecah masalah menjadi bagian-bagian, melihat pola antarbagian, memilih informasi yang relevan, kemudian menyusun algoritma penyelesaian. Urutan penggunaan konsep dapat menyesuaikan karakter persoalan; konsep-konsep tersebut bukan sekadar daftar istilah yang berdiri sendiri.

3. Algoritma

   Algoritma merupakan langkah-langkah atau aturan terstruktur untuk menyelesaikan masalah. Dalam kehidupan sehari-hari, peserta didik menjumpai urutan instruksi melalui resep memasak, petunjuk penggunaan perangkat, atau langkah mengerjakan ujian. Dalam komputer, algoritma perlu disampaikan dengan jelas agar mesin dapat mengikuti instruksi tersebut.

   Algoritma yang baik untuk latihan kelas memiliki tujuan yang jelas, urutan yang logis, langkah yang tidak saling bertentangan, serta keluaran atau kondisi selesai yang dapat dikenali. Ketika meninjau algoritma, peserta didik dapat memeriksa apakah ada langkah yang hilang, urutannya keliru, atau ada istilah yang dapat ditafsirkan dengan lebih dari satu cara.

   Bentuk penulisan algoritma:

| Bentuk | Karakteristik | Kelebihan | Keterbatasan | Kapan berguna |
|---|---|---|---|---|
| Deskriptif | Langkah ditulis menggunakan bahasa sehari-hari. | Paling mudah dibaca oleh orang yang belum mengenal notasi pemrograman. | Pada masalah kompleks dapat menjadi panjang, ambigu, dan sulit diperiksa. | Saat menyampaikan ide awal atau prosedur sederhana. |
| Pseudocode | Langkah penyelesaian ditulis dengan struktur menyerupai kode, tetapi tidak terikat pada satu bahasa pemrograman tertentu. | Lebih terstruktur; kondisi dan pengulangan dapat terlihat jelas. | Memerlukan pemahaman terhadap kata kunci/struktur yang digunakan; tidak langsung dijalankan sebagai program. | Saat menerjemahkan ide menjadi rancangan program. |
| Flowchart | Langkah dan keputusan ditampilkan menggunakan diagram dengan simbol yang dihubungkan alur. | Memudahkan pembaca melihat alur proses dan titik keputusan secara visual. | Diagram dapat menjadi besar dan sulit ditata untuk masalah yang sangat kompleks. | Saat memperlihatkan urutan proses, percabangan, dan aliran data. |

Contoh algoritma sederhana: Contoh berikut merupakan contoh penjelas yang disusun untuk membantu memahami notasi; contoh ini bukan kutipan kode dari slide. Masalah: seseorang ingin menentukan apakah perlu membawa payung berdasarkan informasi cuaca.

a) Bentuk Algoritma Deskriptif

```
1. Mulai.
2. Periksa informasi cuaca.
3. Jika diperkirakan hujan, bawa payung.
4. Jika tidak diperkirakan hujan, payung tidak perlu dibawa.
5. Selesai.
```

b) Bentuk Algoritma Pseudocode

```
MULAI
Baca perkiraan_cuaca
JIKA perkiraan_cuaca = "hujan" MAKA
   Tampilkan "Bawa payung"
JIKA TIDAK
   Tampilkan "Payung tidak diperlukan"
AKHIR JIKA
SELESAI
```

c) Bentuk Flowchart

Mulai (terminator) → masukan perkiraan cuaca (input) → keputusan “diperkirakan hujan?” (decision) → cabang YA menampilkan “bawa payung”; cabang TIDAK menampilkan “payung tidak diperlukan” (display/output) → Selesai (terminator).

4. Simbol Flowchart

Flowchart adalah representasi algoritma menggunakan diagram. Simbol membantu pembaca mengenali jenis aktivitas dalam setiap bagian proses, sedangkan garis panah

menunjukkan arah alur. Simbol yang disebut dalam presentasi dipertahankan pada tabel berikut.

| Simbol/istilah pada materi | Fungsi menurut cakupan materi | Petunjuk pengenalan dalam alur |
|---|---|---|
| Terminator / terminal | Menandai awal atau akhir proses. | Cari titik mulai dan berhenti. |
| Process | Menunjukkan proses atau pengolahan, misalnya operasi hitung. | Biasanya memuat kegiatan yang dilakukan. |
| Decision | Menentukan cabang proses berdasarkan kondisi, umumnya YA/TIDAK. | Periksa pertanyaan/kondisi yang perlu diputuskan. |
| Flow | Menghubungkan simbol dan menunjukkan arah alur. | Ikuti arah panah dari satu langkah ke langkah selanjutnya. |
| Stored data | Melambangkan database atau file data. | Menandai lokasi penyimpanan atau sumber data. |
| Input / output | Menunjukkan informasi masuk atau keluar sistem. | Tanyakan apakah data diterima atau hasil ditampilkan. |
| On-page connector | Menghubungkan bagian alur pada halaman yang sama dan membantu menghindari garis terlalu panjang. | Cari penanda penghubung dengan identitas yang sama. |
| Off-page connector | Menghubungkan alur yang berlanjut pada halaman berbeda. | Cari keterangan rujukan ke halaman lain. |
| Manual operation | Menunjukkan proses yang dilakukan secara manual, bukan oleh komputer. | Periksa siapa atau apa yang melakukan langkah. |
| Delay | Menunjukkan periode penundaan sebagai bagian dari proses. | Cari waktu tunggu atau jeda. |
| Document | Menunjukkan dokumen yang dicetak di kertas. | Menandai dokumen sebagai keluaran/artefak. |
| Alternate process | Menunjukkan proses alternatif dari alur normal jika diperlukan. | Tentukan alur alternatif dan kondisi penggunaannya. |
| Pre-defined process | Menunjukkan proses yang sudah ditentukan di tempat lain. | Proses dirujuk, tidak perlu dirinci ulang di bagian itu. |

| Simbol/istilah pada materi | Fungsi menurut cakupan materi | Petunjuk pengenalan dalam alur |
|---|---|---|
| Display | Menunjukkan perangkat/keluaran yang ditampilkan, misalnya layar. | Cari informasi yang muncul pada layar. |
| Manual input | Menunjukkan data yang dimasukkan secara manual. | Identifikasi informasi yang diketik/dimasukkan pengguna. |
| Preparation | Menunjukkan persiapan/inisialisasi sebelum proses berikutnya. | Periksa nilai atau kondisi yang disiapkan. |

Latihan membaca flowchart:

Ketika melihat diagram, telusuri simbol dari awal sampai selesai. Sebutkan data yang masuk, proses yang dilakukan, keputusan yang mungkin terjadi, dan informasi yang keluar. Jangan hanya menghafal nama simbol tanpa menghubungkannya dengan fungsi.

5. Struktur Dasar Pemrograman C/C++

   Materi presentasi mengarahkan peserta didik untuk memahami pemrograman C/C++ pada Arduino dan membuat program Arduino. Pada tingkat pengantar, peserta didik perlu melihat program sebagai kumpulan instruksi dengan struktur dan aturan penulisan tertentu. Potongan berikut disiapkan sebagai contoh penjelas untuk latihan; guru dapat menyesuaikan dengan contoh visual pada slide.

```
// Contoh kerangka sederhana Arduino
#define LED_PIN 13

void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  Serial.println("LED menyala");
```

```
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  Serial.println("LED mati");
  delay(1000);
}
```

Penjelasan untuk pembelajaran: #define memberi nama pada nilai tetap yang digunakan dalam program; setup() berisi pengaturan awal; loop() berisi instruksi yang dijalankan berulang; pinMode() menetapkan mode pin; digitalWrite() memberikan keadaan digital; Serial.println() menampilkan pesan pada baris baru; delay() menunda proses selama waktu yang ditentukan. Pada contoh, angka pin hanya ilustrasi dan harus disesuaikan dengan rangkaian serta jenis board.

Sintaks dasar:

| Unsur | Makna/fungsi | Contoh/penjelasan |
|---|---|---|
| Block comment | Komentar yang digunakan untuk memberi keterangan pada bagian kode dan tidak menjadi instruksi yang dieksekusi. | /* Keterangan beberapa baris */ |
| Semicolon (;) | Titik koma digunakan untuk mengakhiri banyak pernyataan kode. | digitalWrite(LED_PIN, HIGH); |
| Curly brace { } | Kurung kurawal membatasi awal dan akhir suatu blok pernyataan. | Dipakai pada blok setup(), loop(), if, dan perulangan. |
| #define | Memberi nama pada nilai konstan sebelum program dijalankan/kompilasi. | Contoh: #define LED_PIN 13 |
| #include | Memasukkan library yang diperlukan ke dalam sketch. | Contoh: #include <NamaLibrary.h> jika library benar-benar tersedia. |
| Variabel | Nama untuk tempat menyimpan nilai yang dapat berubah selama program berjalan. | Contoh: int nilaiSensor = 0; |
| Konstanta | Nilai yang ditetapkan dan dipakai sebagai acuan. | Dalam materi: TRUE/FALSE, HIGH/LOW, INPUT/OUTPUT. |

Aturan penamaan variabel:

a) Nama variabel tidak diawali angka. Contoh yang perlu dihindari: 3nilai.
b) Nama variabel tidak menggunakan spasi. Gunakan underscore (_) jika perlu memisahkan kata, misalnya nilai_sensor.
c) Jangan menggunakan istilah yang telah dipakai sebagai nama khusus dalam bahasa/lingkungan Arduino, misalnya main, setup, atau loop.
d) Gunakan nama yang menjelaskan isi nilai agar program lebih mudah dibaca.

6. Konstanta Pemrograman

Pada presentasi, konstanta dibagi menjadi tiga kelompok yang dikenalkan: TRUE/FALSE, HIGH/LOW, dan INPUT/OUTPUT. Kelompok ini tidak memiliki fungsi yang identik: TRUE/FALSE menyatakan kebenaran kondisi, HIGH/LOW menyatakan keadaan logika pada pin digital, sedangkan INPUT/OUTPUT digunakan saat menentukan mode pin melalui pinMode().

| Konstanta | Penjelasan | Contoh cara berpikir |
|---|---|---|
| TRUE / FALSE | TRUE menandakan kondisi benar dan FALSE menandakan kondisi salah. Materi menyebut TRUE sebagai nilai benar dan FALSE sebagai nilai salah. | Jika kondisi yang diperiksa terpenuhi, alur dapat masuk ke cabang TRUE; jika tidak, kondisi bernilai FALSE. |
| HIGH / LOW | Dipakai untuk menyatakan keadaan logika pada pin digital. Pada materi, HIGH dihubungkan dengan ON/1 dan LOW dengan OFF/0. | digitalWrite(LED_PIN, HIGH) meminta pin berada pada kondisi HIGH; LOW meminta kondisi LOW. |
| INPUT / OUTPUT | Digunakan pada pinMode() untuk menetapkan apakah pin dipakai menerima masukan atau memberikan keluaran. | Tombol/sensor dapat berperan sebagai sumber masukan, sedangkan LED sebagai keluaran. |

7. Komunikasi Serial

Serial digunakan untuk komunikasi antara board dan komputer. Presentasi menyebut tiga perintah utama: Serial.begin(), Serial.print(), dan Serial.println(). Peserta didik

perlu membedakan tujuan masing-masing perintah, terutama fungsi memulai komunikasi dan fungsi menampilkan informasi pada Serial Monitor.

| Perintah | Fungsi pada materi | Perbedaan yang perlu dipahami |
|---|---|---|
| Serial.begin(kecepatan) | Mengaktifkan komunikasi serial dengan nilai kecepatan transmisi yang ditentukan. | Umumnya ditempatkan pada bagian pengaturan awal program. |
| Serial.print(data) | Menampilkan data tanpa otomatis berpindah ke baris baru setelah data tersebut. | Beberapa pemanggilan print dapat tampak berurutan pada satu baris. |
| Serial.println(data) | Menampilkan data lalu melanjutkan tampilan ke baris baru. | Memudahkan pembaca memisahkan hasil antarbaris. |

```
void setup() {
  Serial.begin(9600);
  Serial.print("Nilai: ");
  Serial.println(25);
}

void loop() {
}
```

Contoh di atas digunakan untuk membandingkan tampilan print dan println. Peserta didik diminta memperkirakan tampilan sebelum guru menunjukkan atau menjelaskannya. Angka 9600 hanyalah contoh nilai kecepatan komunikasi yang juga tampak pada banyak contoh Arduino; kecocokan pengaturan Serial Monitor perlu mengikuti program yang dijalankan.

8. Pin Digital Analog I/O

| Fungsi | Tujuan | Pertanyaan pengarah | Contoh konseptual |
|---|---|---|---|
| pinMode(pin, mode) | Mengatur pin sebagai INPUT atau OUTPUT. | Apakah pin menerima informasi atau memberikan respons? | Menetapkan pin LED sebagai OUTPUT. |
| digitalRead(pin) | Membaca nilai digital pin dengan hasil HIGH atau LOW. | Nilai digital apa yang terbaca pada pin? | Membaca kondisi tombol yang dirancang sebagai input digital. |
| digitalWrite(pin, nilai) | Menuliskan nilai HIGH atau LOW pada pin digital. | Kondisi digital apa yang ingin diberikan pada pin? | Memerintahkan LED menyala/padam, sesuai rangkaian. |
| analogRead(pin) | Membaca nilai dari pin analog dan menyimpannya ke variabel. | Informasi sensor analog apa yang perlu dibaca? | Membaca perubahan nilai sensor pada pin analog. |
| analogWrite(pin, nilai) | Menghasilkan keluaran dengan metode PWM pada pin yang mendukungnya. | Bagaimana mengubah keluaran melalui nilai yang ditetapkan? | Mengatur tingkat terang LED pada pin PWM yang sesuai. |

Digital I/O dan analog I/O tidak boleh disamakan begitu saja. Fungsi digitalRead() mengembalikan keadaan digital HIGH/LOW, sedangkan analogRead() membaca nilai masukan dari pin analog. analogWrite() pada banyak papan Arduino menghasilkan PWM; meskipun namanya mengandung kata “analog”, sinyalnya tidak sama dengan keluaran tegangan analog sejati pada semua board.

9. Percabangan IF dan IF ELSE

   Percabangan memungkinkan program memilih tindakan berdasarkan kondisi. Presentasi mengenalkan IF untuk menjalankan pernyataan ketika kondisi terpenuhi (TRUE) dan IF-ELSE untuk memilih salah satu dari dua jalur: jalur IF ketika kondisi benar, atau jalur ELSE ketika kondisi salah.

```
if (nilaiSensor > 500) {
```

```
  Serial.println("Nilai tinggi");
}
```

Pada contoh pertama, pesan hanya ditampilkan apabila nilaiSensor lebih besar dari 500. Jika kondisi tidak terpenuhi, tidak ada pernyataan else yang dijalankan.

```
if (nilaiSensor > 500) {
  Serial.println("Nilai tinggi");
} else {
  Serial.println("Nilai rendah atau sama dengan 500");
}
```

Pada contoh kedua, program memiliki dua kemungkinan respons. Guru dapat mengubah batas kondisi atau nilai masukan untuk mengajak peserta didik memprediksi cabang mana yang berjalan. Contoh ini merupakan penjelas yang disusun untuk pembelajaran, bukan kutipan langsung dari berkas slide.

10. Perulangan FOR, WHILE, DO WHILE

    Perulangan digunakan ketika instruksi perlu dilakukan lebih dari satu kali. Bentuk perulangan yang dipresentasikan mempunyai karakteristik berbeda, sehingga peserta didik perlu memahami kondisi yang diperiksa dan waktu pemeriksaan tersebut.

| Struktur | Karakteristik sesuai materi | Hal yang ditelusuri peserta didik |
|---|---|---|
| for | Digunakan untuk perulangan yang bersifat terbatas. Sintaks memuat nilai awal, kondisi, dan penambahan (increment) atau pengurangan (decrement). Increment dapat menggunakan ++ dan decrement menggunakan --. | Nilai awal, kondisi berhenti, perubahan nilai, dan banyaknya putaran. |
| while | Melakukan perulangan selama kondisi terpenuhi dan bernilai TRUE. | Kondisi diperiksa sebelum badan perulangan dijalankan; kondisi perlu berubah agar perulangan dapat berhenti. |

| Struktur | Karakteristik sesuai materi | Hal yang ditelusuri peserta didik |
|---|---|---|
| do-while | Menjalankan pernyataan terlebih dahulu kemudian memeriksa kondisi while. Jika kondisi sesuai, pernyataan dijalankan kembali; jika tidak sesuai, pengulangan berhenti. | Badan perulangan dijalankan setidaknya satu kali sebelum kondisi pertama kali diperiksa. |

```
for (int i = 0; i < 3; i++) {
  Serial.println(i);
}
```

Untuk contoh for tersebut, peserta didik dapat menelusuri urutan nilai i yang dicetak: dimulai dari 0, diulang selama i lebih kecil dari 3, lalu i bertambah satu setiap putaran. Guru tidak perlu menekankan hafalan sintaks saja; fokuskan perhatian pada cara kerja nilai awal, kondisi, dan perubahan.

```
int i = 0;
while (i < 3) {
  Serial.println(i);
  i++;
}
```

Contoh while memperlihatkan pentingnya memperbarui variabel i di dalam perulangan. Jika nilai i tidak berubah padahal kondisi tetap benar, program dapat terus mengulang. Hal ini menjadi alasan penting untuk membaca badan perulangan secara teliti.

```
int i = 0;
do {
  Serial.println(i);
  i++;
} while (i < 3);
```

Pada do-while, pernyataan dijalankan terlebih dahulu, baru kemudian kondisi diperiksa. Peserta didik dapat membandingkan pola ini dengan while untuk melihat perbedaan waktu pemeriksaan kondisi.

## K. RENCANA PEMBELAJARAN PERTEMUAN 1

Alokasi waktu: 3 JP × 45 menit = 135 menit. Fokus: berpikir komputasional, empat konsep utama, struktur data sebagai pengantar, pengertian algoritma, tiga bentuk representasi algoritma, dan simbol flowchart.

1. Tujuan Pertemuan
    a) Peserta didik mampu menjelaskan berpikir komputasional dengan kalimat sendiri.
    b) Peserta didik mampu membedakan dekomposisi, abstraksi, algoritma, dan pengenalan pola melalui satu masalah kontekstual.
    c) Peserta didik mampu membandingkan representasi deskriptif, pseudocode, dan flowchart.
    d) Peserta didik mampu mengenali fungsi simbol flowchart yang disajikan pada materi.
    e) Peserta didik berani menjelaskan alasan dan merevisi gagasan berdasarkan informasi yang dipelajari.

2. Persiapan Sebelum Kelas
    a) Guru memeriksa bahwa slide pengantar, struktur data, berpikir komputasional, algoritma, notasi algoritma, dan simbol flowchart dapat ditampilkan dengan jelas.
    b) Guru menyiapkan satu masalah kontekstual sederhana, misalnya menyusun langkah persiapan presentasi kelompok atau mengatur proses peminjaman buku.
    c) Guru menyiapkan papan tulis untuk membandingkan empat konsep berpikir komputasional.
    d) Guru menyiapkan LKPD Latihan 1 untuk digunakan sebagai latihan terbimbing setelah konsep awal dibahas. LKPD ini bukan lembar tes.

e) Guru menyiapkan catatan observasi untuk merekam keaktifan bertanya, kualitas alasan, dan kemampuan menjelaskan proses.

3. Langkah-Langkah Pembelajaran

| Waktu | Tahap Inquiry / kegiatan | Aktivitas guru | Aktivitas peserta didik dan bukti belajar |
|---|---|---|---|
| 0–5 menit | Pembukaan dan kesiapan belajar | Menyapa, mengecek kehadiran/kesiapan, memastikan tampilan materi siap, dan menyampaikan tujuan dengan bahasa sederhana. | Menyiapkan alat belajar, menyimak tujuan, dan menjawab pertanyaan pembuka. |
| 5–15 menit | Orientasi / stimulasi | Menampilkan masalah "Bagaimana menyelenggarakan presentasi kelompok agar siap tepat waktu?" Guru meminta peserta didik menceritakan pengalaman dan menunjukkan bahwa masalah dapat diurai. | Mengamati masalah, menyebutkan hal yang diketahui, dan mengaitkan dengan pengalaman sekolah. |
| 15–25 menit | Merumuskan masalah | Membimbing pertanyaan: bagian pekerjaan apa saja yang diperlukan; informasi mana yang penting; apakah ada pola tugas yang berulang; bagaimana urutannya? | Menyampaikan pertanyaan yang ingin dijawab dan merumuskan satu masalah fokus. |
| 25–35 menit | Merumuskan dugaan | Meminta peserta didik memperkirakan strategi yang paling membantu sebelum definisi diberikan. Menuliskan kata kunci jawaban di papan. | Menyampaikan dugaan awal dan alasan, misalnya membagi tugas atau menyusun urutan kerja. |
| 35–55 menit | Mengumpulkan informasi: berpikir komputasional | Memandu pembacaan salindia tentang pengertian berpikir komputasional, struktur data, dan empat konsep utamanya. Memberi contoh tambahan jika definisi sulit dibedakan. | Mencatat istilah, menandai bagian yang belum jelas, dan menghubungkan definisi dengan masalah awal. |

| Waktu | Tahap Inquiry / kegiatan | Aktivitas guru | Aktivitas peserta didik dan bukti belajar |
|---|---|---|---|
| 55–70 menit | Menguji dugaan konsep | Mengajukan situasi pembanding untuk menguji apakah peserta didik dapat menunjukkan dekomposisi, abstraksi, pengenalan pola, dan algoritma. Meluruskan miskonsepsi melalui pertanyaan, bukan langsung memberi semua jawaban. | Mengklasifikasikan tindakan dalam situasi, memberi alasan, dan memperbaiki pemahaman jika ada bukti yang berbeda. |
| 70–85 menit | Mengumpulkan informasi: algoritma dan notasi | Menjelaskan pengertian algoritma, lalu menampilkan notasi deskriptif, pseudocode, dan flowchart. Menekankan perbedaan bentuk dan kegunaan. | Membandingkan cara penyajian langkah; mengamati contoh dan mencatat kelebihan/keterbatasan setiap notasi. |
| 85–105 menit | Latihan inquiry terbimbing | Mengarahkan LKPD Latihan 1 untuk menelaah satu persoalan dan menuliskan urutannya dalam deskripsi/pseudocode atau merancang flowchart sederhana. Menjawab pertanyaan tanpa menjadikan LKPD sebagai tes. | Mengerjakan latihan secara berpasangan/kelompok kecil, mengajukan pertanyaan, dan menjelaskan pilihan representasi. |
| 105–120 menit | Mengumpulkan dan menguji informasi: simbol flowchart | Menampilkan simbol flowchart pada slide; meminta peserta didik menghubungkan nama simbol dengan fungsi serta mengikuti arah alur pada contoh. | Mengidentifikasi simbol, menelusuri alur, dan menjelaskan bagaimana simbol keputusan mengubah jalur. |
| 120–128 menit | Merumuskan kesimpulan dan refleksi | Meminta beberapa peserta didik menyampaikan kesimpulan. Menyusun rangkuman: berpikir komputasional, algoritma, representasi, dan fungsi simbol. | Menyampaikan satu konsep yang dipahami, satu contoh penggunaannya, dan satu hal yang masih membingungkan. |

| Waktu | Tahap Inquiry / kegiatan | Aktivitas guru | Aktivitas peserta didik dan bukti belajar |
|---|---|---|---|
| 128–135 menit | Penutup | Menyampaikan pratinjau pertemuan kedua yang menghubungkan algoritma dengan instruksi C/C++ Arduino. Memberikan apresiasi terhadap pertanyaan dan perbaikan pemahaman. | Menyimak rangkuman, menyiapkan pertanyaan lanjutan, dan mengakhiri pembelajaran dengan tertib. |

4. Pertanyaan Pemandu

| Pertanyaan guru | Arah respons yang diharapkan | Tindak lanjut guru |
|---|---|---|
| “Apa bedanya memecah pekerjaan dan menyusun langkah?” | Memecah pekerjaan menunjukkan dekomposisi; menyusun urutan tindakan menunjukkan algoritma. | Tunjukkan kedua konsep dengan contoh masalah yang sama. |
| “Apakah detail warna kertas presentasi selalu penting?” | Tidak selalu; kepentingannya bergantung pada tujuan dan kondisi masalah. | Hubungkan dengan abstraksi: memilih informasi relevan. |
| “Bagaimana kita tahu dua tugas memiliki pola yang serupa?” | Ada struktur atau langkah yang berulang/serupa walaupun detailnya berbeda. | Minta peserta didik menyebut bagian yang sama dan yang berbeda. |
| “Mengapa bahasa sehari-hari dapat menimbulkan ambiguitas?” | Kata-kata dapat ditafsirkan berbeda atau terlalu umum. | Bandingkan instruksi yang kabur dengan langkah yang lebih spesifik. |
| “Kapan flowchart lebih membantu dibandingkan deskripsi?” | Saat perlu melihat alur, keputusan, dan cabang proses secara visual. | Mintalah peserta didik menelusuri jalur YA/TIDAK. |
| “Apa yang dilakukan simbol decision?” | Menunjukkan pilihan alur berdasarkan suatu kondisi. | Berikan satu kondisi tambahan untuk melihat apakah peserta didik dapat menelusuri cabang. |

5. Catatan Penguatan Guru

   Jika waktu tidak cukup, pastikan seluruh simbol flowchart tetap diperkenalkan tetapi latihan klasifikasi dapat dilakukan secara lisan. Jangan mengorbankan pemahaman empat konsep utama dan perbedaan tiga representasi algoritma. Latihan flowchart yang lebih kompleks dapat menjadi pengayaan atau kegiatan pertemuan lanjutan.

## L. RENCANA PEMBELAJARAN PERTEMUAN 2

Alokasi waktu: 2 JP × 45 menit = 90 menit. Fokus: sintaks dasar C/C++ Arduino, variabel dan konstanta, komunikasi serial, pin digital/analog I/O, percabangan if/if-else, dan perulangan for/while/do-while.

1. Tujuan Pertemuan
    a) Peserta didik mampu mengenali unsur dasar sintaks dan menjelaskan kegunaan komentar, semicolon, curly brace, #define, dan #include.
    b) Peserta didik mampu menjelaskan variabel, aturan penamaan sederhana, dan kelompok konstanta pada materi.
    c) Peserta didik mampu membedakan fungsi komunikasi serial dan fungsi pin I/O.
    d) Peserta didik mampu menjelaskan cara kerja percabangan dan perulangan pada contoh kode pendek.
    e) Peserta didik mampu melakukan latihan penelusuran kode dengan menyatakan dugaan, membandingkan dugaan dengan struktur program, dan merumuskan kesimpulan.

2. Persiapan Sebelum Kelas
    a) Guru menyiapkan slide struktur dasar Arduino, sintaks, variabel, konstanta, serial, pin I/O, if/if-else, serta perulangan.
    b) Guru memilih potongan kode pendek yang dapat dibaca dari layar dan menyiapkan versi cetaknya bila ukuran kelas/layar tidak memungkinkan.
    c) Guru menyiapkan LKPD Latihan 2 sebagai latihan membaca kode dan mengelompokkan fungsi; lembar tersebut tidak digunakan untuk tes atau asesmen sumatif.
    d) Guru meninjau kembali perbedaan pin I/O berdasarkan jenis board. Jika contoh analogWrite dibahas pada Arduino Uno, gunakan pin PWM yang sesuai dengan dokumentasi resmi board.
    e) Guru menyiapkan rubrik observasi terpisah untuk mencatat kemampuan menjelaskan fungsi sintaks dan menelusuri alur program.

3. Langkah-Langkah Pembelajaran

| Waktu | Tahap Inquiry / kegiatan | Aktivitas guru | Aktivitas peserta didik dan bukti belajar |
|---|---|---|---|
| 0–5 menit | Pembukaan dan pengaitan | Menyapa, meninjau singkat kesimpulan pertemuan pertama, dan menghubungkan algoritma dengan instruksi yang dapat dijalankan perangkat. | Mengingat kembali algoritma dan menjelaskan mengapa instruksi perlu jelas. |
| 5–10 menit | Orientasi / stimulasi | Menampilkan potongan kode pendek dengan titik koma/kurung kurawal yang perlu diperhatikan. Menanyakan apa yang membuat kode dapat dibaca secara terstruktur. | Mengamati kode dan menyampaikan dugaan tentang fungsi tanda atau kata kunci. |
| 10–15 menit | Merumuskan masalah dan dugaan | Mengajak peserta didik merumuskan pertanyaan: bagaimana kode mengatur pin, menampilkan informasi, memilih tindakan, dan mengulang instruksi? | Mengajukan pertanyaan dan menyampaikan dugaan awal. |
| 15–32 menit | Mengumpulkan informasi | Menjelaskan kerangka sketch, komentar, semicolon, curly brace, #define, #include, variabel, dan konstanta. Menampilkan tabel perbandingan. | Menandai unsur kode, memberi contoh nama variabel yang sesuai, dan membedakan TRUE/FALSE, HIGH/LOW, INPUT/OUTPUT. |
| 32–45 menit | Mengumpulkan informasi: serial dan I/O | Menjelaskan Serial.begin(), print(), println(), pinMode(), digitalRead(), digitalWrite(), analogRead(), dan analogWrite() dengan contoh yang relevan. | Memilih fungsi yang sesuai dengan kebutuhan pada situasi yang disampaikan guru dan menjelaskan alasannya. |
| 45–55 menit | Menguji dugaan: percabangan | Menampilkan contoh if dan if-else. Mengajak peserta didik memprediksi keluaran berdasarkan kondisi, lalu membahas cabang yang dijalankan. | Menelusuri kondisi TRUE/FALSE, memperkirakan keluaran, dan menjelaskan dasar pilihannya. |

| Waktu | Tahap Inquiry / kegiatan | Aktivitas guru | Aktivitas peserta didik dan bukti belajar |
|---|---|---|---|
| 55–65 menit | Menguji dugaan: perulangan | Membandingkan for, while, dan do-while; menelusuri nilai awal, kondisi, dan perubahan nilai pada satu contoh sederhana. | Menyebutkan perbedaan struktur; memprediksi nilai yang muncul dan alasan berhentinya pengulangan. |
| 65–78 menit | Latihan inquiry terbimbing | Mengarahkan LKPD Latihan 2. Guru berkeliling, memfasilitasi tanya jawab, dan memberi petunjuk bertahap. Latihan bukan tes dan tidak dijadikan satu-satunya dasar penilaian. | Membaca kode, mengelompokkan fungsi, memperbaiki satu kesalahan sederhana, serta mendiskusikan hasil dengan teman. |
| 78–85 menit | Kesimpulan dan refleksi | Mengajak peserta didik menyusun rangkuman hubungan algoritma, sintaks, input/output, kondisi, dan perulangan. Mengklarifikasi miskonsepsi penting. | Menyebut konsep paling penting, contoh fungsi yang dipahami, dan hal yang perlu dipelajari lebih lanjut. |
| 85–90 menit | Penutup | Memberi apresiasi, menyampaikan tindak lanjut, dan mengingatkan bahwa kemampuan coding tumbuh melalui latihan bertahap. | Menyimak rangkuman akhir dan menyampaikan satu pertanyaan/komitmen belajar. |

4. Prioritas Isi

   Karena materi pemrograman mencakup banyak istilah dalam 90 menit, guru membedakan kedalaman yang diharapkan. Semua kelompok fungsi dikenalkan secara bermakna, tetapi latihan intensif difokuskan pada: (a) memahami peran sintaks dasar; (b) membedakan fungsi menerima data dan mengirim keluaran; (c) menelusuri satu contoh if-else; dan (d) menjelaskan perubahan nilai pada satu contoh perulangan. Peserta didik yang lebih siap dapat diminta membandingkan tiga jenis loop atau menulis ulang algoritma menjadi potongan kode pendek.

5. Pertanyaan Pemandu

| Pertanyaan guru | Arah respons yang diharapkan | Tindak lanjut guru |
|---|---|---|
| “Mengapa pernyataan kode sering diakhiri titik koma?” | Titik koma menandai akhir banyak pernyataan. | Tunjukkan satu baris instruksi dan minta peserta didik menunjuk akhir pernyataan. |
| “Apa perbedaan variabel dan konstanta?” | Variabel menyimpan nilai yang dapat berubah; konstanta/nilai tetap dipakai sebagai acuan. | Minta peserta didik menyebut contoh nama variabel yang bermakna. |
| “Fungsi mana yang digunakan untuk membaca nilai pin digital?” | digitalRead(). | Bandingkan dengan digitalWrite() agar tidak tertukar. |
| “Apa perbedaan print dan println?” | println memindahkan keluaran ke baris baru; print tidak secara otomatis melakukan itu. | Minta peserta didik memperkirakan bentuk dua pesan yang ditampilkan. |
| “Apa yang terjadi jika kondisi if bernilai FALSE dan tidak ada else?” | Isi blok if tidak dijalankan; program melanjutkan ke instruksi berikutnya. | Bandingkan dengan if-else. |
| “Apa risiko perulangan while jika kondisi tidak pernah berubah?” | Kondisi dapat terus bernilai TRUE sehingga program berulang tanpa berhenti. | Tunjukkan pentingnya perubahan nilai/kondisi berhenti. |
| “Apa perbedaan kapan kondisi diperiksa pada while dan do-while?” | while memeriksa sebelum badan perulangan; do-while menjalankan badan dulu lalu memeriksa kondisi. | Minta peserta didik menelusuri contoh singkat. |

## M. ASESMEN PEMBELAJARAN

1. Asesmen Diagnostik

   Pada awal setiap pertemuan, guru mengajukan pertanyaan lisan singkat untuk mengetahui pengetahuan awal. Jawaban ini digunakan untuk menyesuaikan penjelasan dan dukungan, bukan sebagai tes tertulis.

| Pertemuan | Pertanyaan lisan | Informasi yang dicari guru |
|---|---|---|
| 1 | “Bagaimana kamu menyelesaikan pekerjaan besar yang memiliki banyak langkah?” | Apakah peserta didik memahami gagasan memecah tugas dan menyusun urutan. |

| Pertemuan | Pertanyaan lisan | Informasi yang dicari guru |
|---|---|---|
| 1 | “Apa yang membuat instruksi mudah atau sulit dipahami orang lain?” | Pemahaman awal tentang kejelasan algoritma dan kemungkinan ambiguitas. |
| 2 | “Apakah kamu pernah melihat kode atau instruksi untuk perangkat elektronik?” | Pengalaman awal terhadap kode dan kebutuhan contoh konkret. |
| 2 | “Apa yang kamu duga dilakukan komputer ketika diberi kondisi benar atau salah?” | Pemahaman awal tentang keputusan/kondisi dan alur program. |

2. Asesmen Formatif
    a) Observasi keterlibatan: apakah peserta didik mengamati contoh, bertanya, menyampaikan dugaan, dan menanggapi pendapat teman.
    b) Pertanyaan lisan: apakah peserta didik dapat memberikan alasan dan membandingkan konsep dengan benar.
    c) Penelusuran kode bersama: apakah peserta didik dapat mengikuti kondisi, aliran instruksi, dan perubahan nilai dalam loop.
    d) Pemeriksaan pemahaman secara informal: guru meminta peserta didik menyampaikan ulang konsep dengan bahasa sendiri dan memberikan umpan balik langsung.
    e) Refleksi akhir: peserta didik menyebutkan hal yang sudah dipahami dan hal yang masih memerlukan bantuan.

3. Bukti Pembelajaran

| Bukti | Cara memperolehnya | Penggunaan oleh guru |
|---|---|---|
| Penjelasan lisan | Pertanyaan dan diskusi kelas. | Mengidentifikasi konsep yang masih tertukar dan menentukan penguatan. |
| Catatan observasi | Guru mencatat proses bertanya, memberi alasan, menelusuri contoh, dan berkolaborasi. | Menentukan peserta didik yang memerlukan pertanyaan penuntun atau pengayaan. |
| Hasil latihan LKPD | Melihat cara peserta didik mencoba strategi dan memperbaiki jawaban. | Memberi umpan balik belajar; tidak digunakan sebagai lembar tes/sumatif. |

| Bukti | Cara memperolehnya | Penggunaan oleh guru |
|---|---|---|
| Refleksi peserta didik | Jawaban lisan atau refleksi singkat pada akhir pertemuan. | Menentukan tindak lanjut dan kebutuhan penjelasan berikutnya. |

## N. DIFERENSIASI DAN DUKUNGAN BELAJAR

| Aspek | Dukungan dasar | Penyesuaian untuk peserta didik yang membutuhkan dukungan | Pengayaan untuk peserta didik yang lebih siap |
|---|---|---|---|
| Konten | Penjelasan konsep dilengkapi contoh keseharian dan salindia. | Guru menjelaskan istilah satu per satu, menggunakan kalimat pendek, dan menekankan kata kunci. | Peserta didik membandingkan contoh yang lebih kompleks atau menghubungkan konsep dengan kasus lain. |
| Proses | Diskusi kelas, kerja berpasangan, latihan terbimbing, dan tanya jawab. | Gunakan think-pair-share, petunjuk langkah demi langkah, serta waktu tambahan untuk menalar. | Peserta didik menjelaskan alternatif solusi, menemukan kelemahan algoritma, atau menyederhanakan representasi. |
| Produk/bukti belajar | Penjelasan lisan, notasi algoritma, klasifikasi fungsi, dan refleksi. | Peserta didik dapat menjelaskan secara lisan sebelum menuliskan jawaban atau memakai template. | Peserta didik dapat membuat variasi pseudocode, menambahkan kondisi, atau membandingkan while dengan do-while. |
| Lingkungan | Lingkungan aman untuk bertanya dan mengoreksi gagasan. | Tempatkan bersama teman yang suportif; guru mengecek pemahaman secara berkala. | Berikan peran sebagai penjelas konsep atau pemeriksa alur dengan tetap menghargai semua anggota. |

Prinsip inklusif:

Perbedaan jawaban awal tidak digunakan untuk memberi label kemampuan secara permanen. Guru menilai perkembangan dan kualitas alasan, serta menyediakan berbagai jalur untuk memahami materi: mendengar penjelasan, mengamati representasi visual, berdiskusi, membaca contoh, dan mencoba latihan. Peserta didik tidak harus langsung menulis program lengkap untuk menunjukkan bahwa mereka mulai memahami logika program.

## O. REFLEKSI PESERTA DIDIK

Refleksi Pertemuan 1:

1) Konsep berpikir komputasional mana yang paling mudah saya pahami? Apa contohnya?
2) Bagian mana yang masih tertukar antara dekomposisi, abstraksi, pengenalan pola, dan algoritma?
3) Notasi mana yang paling membantu saya menjelaskan langkah: deskriptif, pseudocode, atau flowchart? Mengapa?
4) Simbol flowchart mana yang masih sulit saya kenali?
5) Pertanyaan apa yang ingin saya bawa ke pertemuan berikutnya?

Refleksi Pertemuan 2:

1) Fungsi Arduino mana yang sekarang saya pahami dan dapat saya jelaskan dengan kalimat sendiri?
2) Apa perbedaan fungsi digitalRead() dan digitalWrite()?
3) Bagaimana saya menjelaskan perbedaan if-else, for, while, dan do-while?
4) Bagian kode apa yang paling membingungkan bagi saya, dan bantuan apa yang saya perlukan?
5) Bagaimana cara berpikir komputasional membantu saya memahami program?

## P. REFLEKSI GURU

| Fokus refleksi | Pertanyaan evaluasi guru | Catatan tindak lanjut |
|---|---|---|
| Ketercapaian tujuan | Apakah mayoritas peserta didik dapat menjelaskan konsep utama dengan kata-kata sendiri? Indikator mana yang belum terlihat? | Catat konsep yang memerlukan pengulangan atau contoh tambahan. |
| Efektivitas inquiry | Apakah pertanyaan pemantik menimbulkan rasa ingin tahu dan menghasilkan dugaan yang dapat diuji? | Ubah pertanyaan agar lebih kontekstual atau lebih terarah. |
| Partisipasi | Apakah semua peserta didik mendapat kesempatan untuk menyampaikan pendapat? Siapa yang masih pasif? | Gunakan diskusi berpasangan, waktu berpikir mandiri, atau giliran berbicara. |
| Pengelolaan waktu | Apakah bagian penjelasan dan latihan seimbang? Apakah ada materi yang terlalu cepat? | Prioritaskan indikator inti dan jadikan materi tertentu sebagai pengayaan. |
| Pemahaman kode | Apakah peserta didik mampu menelusuri kondisi dan perulangan, bukan hanya menghafal fungsi? | Siapkan contoh kode lebih pendek, penanda warna, atau penelusuran baris demi baris. |
| Kesesuaian sumber | Apakah contoh dan istilah dalam modul selaras dengan salindia dan board yang digunakan? | Klarifikasi perbedaan/ketidaktepatan teknis dengan transparan. |

## Q. TINDAK LANJUT

1. Penguatan/Remedial

   Peserta didik yang belum mantap memahami materi mendapat pembelajaran ulang yang lebih kecil cakupannya, bukan sekadar mengulang seluruh salindia. Guru memilih fokus berdasarkan bukti observasi dan refleksi.

   a) Jika empat konsep berpikir komputasional masih tertukar, gunakan satu situasi saja dan minta peserta didik menunjukkan bagian masalah, informasi penting, pola, dan urutan penyelesaian.

   b) Jika notasi algoritma belum dipahami, berikan algoritma deskriptif terlebih dahulu, kemudian bantu mengubahnya menjadi pseudocode sebelum menggambar flowchart.

c) Jika simbol flowchart belum dikuasai, gunakan kartu nama-simbol-fungsi dan latihan mencocokkan secara lisan.
d) Jika sintaks Arduino membingungkan, mulai dari kode 3–5 baris dan fokus pada komentar, titik koma, dan kurung kurawal.
e) Jika fungsi I/O masih tertukar, gunakan tabel keputusan: “menerima nilai atau mengirim nilai?”, lalu hubungkan dengan digitalRead/digitalWrite/analogRead/analogWrite.
f) Jika kondisi/perulangan belum dipahami, telusuri satu contoh tanpa perangkat, tulis nilai variabel pada tiap putaran, dan tandai kapan proses berhenti.

2. Pengayaan
   a) Peserta didik membuat algoritma untuk persoalan berbeda dan memilih representasi yang paling sesuai, disertai alasan.
   b) Peserta didik membandingkan dua flowchart yang menyelesaikan masalah sama dan menjelaskan mana yang lebih mudah dibaca.
   c) Peserta didik menambahkan cabang alternatif pada contoh if-else dan memprediksi keluaran untuk beberapa nilai masukan.
   d) Peserta didik membandingkan perulangan for, while, dan do-while pada contoh jumlah putaran yang sama.
   e) Jika perangkat tersedia, peserta didik dapat mencoba potongan program pada simulator atau board dengan pendampingan dan memperhatikan keselamatan rangkaian.

3. Rencana Penguatan

   Dua pertemuan ini merupakan pengenalan dan penguatan konsep. Kegiatan lanjutan dapat diarahkan pada penyusunan algoritma lebih kompleks, praktik Arduino sederhana, atau implementasi dasar sesuai ketersediaan waktu dan sumber belajar. Karena presentasi yang dilampirkan tidak menyediakan uraian secara rinci, kegiatan simulasi coding C/C++ perlu dilengkapi dengan sumber dan panduan tersendiri sebelum dijadikan fokus pembelajaran.

## R. LKPD LATIHAN 1 – MEMECAHKAN MASALAH BERBASIS BERPIKIR KOMPUTASIONAL DAN ALGORITMA

Mata Pelajaran :

Kelas :

Waktu Latihan :

Nama Peserta Didik :

Tanggal :

Tujuan Latihan:

Melalui latihan ini, peserta didik mencoba menguraikan masalah, memilih informasi yang penting, mengenali pola, menyusun algoritma, dan membandingkan representasi algoritma.

Situasi Latihan:

Kelompokmu akan mempresentasikan hasil diskusi di depan kelas. Agar presentasi berjalan lancar, kelompok perlu menyiapkan topik, membagi tugas, menyiapkan bahan/media, berlatih, dan menyepakati urutan penyampaian. Gunakan situasi ini untuk mengerjakan latihan berikut.

Latihan A – Mengurai Masalah

| |
|---|
| Tuliskan minimal empat pekerjaan kecil yang perlu diselesaikan agar presentasi kelompok siap dilakukan? |
| Informasi apa yang paling penting untuk memastikan presentasi siap tepat waktu? Mengapa informasi tersebut penting? |
| Jika kelompok melakukan kegiatan presentasi setiap minggu, bagian pekerjaan apa yang kemungkinan membentuk pola berulang? |

Latihan B – Menulis Algoritma

Tuliskan urutan persiapan presentasi menggunakan bahasa sehari-hari. Pastikan langkah pertama, proses, dan akhir kegiatan terlihat jelas!

Sekarang pilih tiga sampai lima langkah penting dan tuliskan dalam bentuk pseudocode sederhana. Kamu boleh memakai kata MULAI, BACA/PERIKSA, LAKUKAN, JIKA, TAMPILKAN, dan SELESAI sesuai kebutuhan.

Latihan C – Presentasi

Menurutmu, apakah masalah persiapan presentasi lebih mudah dijelaskan dengan deskripsi, pseudocode, atau flowchart? Jelaskan alasanmu.

Simbol flowchart apa yang cocok untuk menunjukkan awal/akhir, proses, dan keputusan? Tuliskan fungsi masing-masing dengan kalimat sendiri.

Diskusi dan Rfleksi:

Bandingkan hasilmu dengan teman. Bagian mana yang berbeda? Apakah perbedaannya membuat algoritma lebih jelas atau justru lebih membingungkan? Perbaiki jawaban berdasarkan alasan yang kamu anggap kuat.

## S. LKPD LATIHAN 2 – MEMBACA LOGIKA DASAR ARDUINO

Mata Pelajaran :

Kelas :

Waktu Latihan :

Nama Peserta Didik :

Tanggal :

Tujuan Latihan:

Peserta didik berlatih mengenali sintaks, membedakan fungsi komunikasi serial dan pin I/O, serta menelusuri percabangan dan perulangan dari potongan kode sederhana.

Latihan A – Mengenali Unsur Kode

```
// Mengatur pin LED

#define LED_PIN 13



void setup() {

  pinMode(LED_PIN, OUTPUT);

  Serial.begin(9600);

}



void loop() {

  digitalWrite(LED_PIN, HIGH);
```

```
  Serial.println("LED menyala");

}
```

Bagian mana yang merupakan komentar? Apa tujuan komentar tersebut?

Apa kegunaan #define pada contoh ini?

Pada bagian mana terlihat kurung kurawal dan titik koma? Mengapa penulisannya perlu diperhatikan?

Fungsi apa yang mengatur pin, mengaktifkan komunikasi serial, memberi keluaran digital, dan menampilkan pesan?

Latihan B – Memilih Fungsi

Tuliskan fungsi yang paling sesuai untuk kebutuhan berikut, lalu jelaskan alasanmu:

| Kebutuhan | Fungsi yang dipilih | Alasan |
|---|---|---|
| Mengatur pin menjadi OUTPUT. | | |
| Membaca nilai dari pin digital. | | |
| Memberi nilai HIGH atau LOW ke pin digital. | | |
| Membaca nilai dari pin analog. | | |

| Kebutuhan | Fungsi yang dipilih | Alasan |
|---|---|---|
| Menampilkan data lalu pindah ke baris baru. | | |
| Menampilkan data tanpa otomatis pindah ke baris baru. | | |

Latihan C – Menelusuri Percabangan

```
int nilai = 650;

if (nilai > 500) {

  Serial.println("Tinggi");

} else {

  Serial.println("Rendah atau sama dengan batas");

}
```

Menurut dugaanmu, pesan apa yang akan ditampilkan? Apa alasanmu?

Jika nilai diubah menjadi 300, apakah pesan berubah? Jelaskan bagaimana kondisi menentukan jalur yang dipilih.

Latihan D – Menelusuri Perulangan

```
for (int i = 0; i < 3; i++) {
  Serial.println(i);
}
```

Nilai apa saja yang diperkirakan ditampilkan? Tuliskan urutannya.

Apa peran nilai awal, kondisi i < 3, dan i++ pada contoh?

Dalam kalimatmu sendiri, jelaskan satu perbedaan penting antara while dan do-while.

Latihan E – Membandingkan While dan Do While

```
int kondisi = 5;
while (kondisi < 5) {
  Serial.println("while dijalankan");
}
do {
  Serial.println("do-while dijalankan");
} while (kondisi < 5);
```

Pesan mana yang diperkirakan muncul? Jelaskan alasanmu dengan memperhatikan kapan kondisi diperiksa

Apa pelajaran yang kamu peroleh dari perbandingan ini ketika memilih struktur perulangan?

Diskusi dan Refleksi:

Bagian kode mana yang paling mudah kamu pahami? Bagian mana yang memerlukan bantuan? Gunakan jawaban ini untuk meminta penjelasan yang lebih spesifik kepada guru, bukan untuk membandingkan diri dengan teman.

## T. INSTRUMEN OBSERVASI GURU

Instrumen ini digunakan guru sebagai catatan proses belajar. Instrumen tidak ditempelkan sebagai soal di dalam LKPD dan tidak dimaksudkan menggantikan kebijakan asesmen sekolah.

1. Rubrik Observasi

| Aspek | 4 — Sangat berkembang | 3 — Berkembang sesuai harapan | 2 — Mulai berkembang | 1 — Perlu bimbingan intensif |
|---|---|---|---|---|
| Merumuskan pertanyaan | Mengajukan pertanyaan fokus, relevan, dan menggali alasan. | Mengajukan pertanyaan relevan tentang materi. | Pertanyaan masih umum atau perlu dipancing. | Belum mengajukan pertanyaan atau belum memahami masalah tanpa pendampingan. |

| Aspek | 4 — Sangat berkembang | 3 — Berkembang sesuai harapan | 2 — Mulai berkembang | 1 — Perlu bimbingan intensif |
|---|---|---|---|---|
| Menyampaikan dugaan dan alasan | Dugaan jelas, disertai alasan yang logis dan dapat diperiksa. | Dugaan jelas dengan alasan sederhana. | Dugaan ada tetapi alasan belum jelas. | Kesulitan menyampaikan dugaan meskipun sudah diberi contoh. |
| Menggunakan informasi | Menghubungkan informasi dari materi dengan masalah secara mandiri. | Menggunakan informasi yang relevan dengan sedikit arahan. | Memerlukan arahan berulang untuk menemukan informasi penting. | Belum dapat memilih informasi relevan meskipun dibimbing. |
| Menguji dan memperbaiki pemahaman | Menguji dugaan, mengidentifikasi kekeliruan, dan memperbaiki alasan dengan jelas. | Dapat memeriksa dugaan dan menerima umpan balik. | Mau memperbaiki jawaban tetapi masih memerlukan contoh langsung. | Belum dapat meninjau ulang jawaban meskipun dibimbing. |
| Komunikasi dan kolaborasi | Menjelaskan gagasan runtut, mendengar pendapat lain, dan membantu diskusi. | Menjelaskan gagasan dan bekerja sama dengan baik. | Berpartisipasi sesekali atau penjelasannya belum runtut. | Jarang berpartisipasi dan memerlukan dukungan untuk terlibat. |
| Konsep algoritma/kode | Dapat menerapkan konsep secara tepat serta memberi alasan atas pilihannya. | Memahami sebagian besar konsep dan menelusuri contoh sederhana. | Masih tertukar pada beberapa istilah/fungsi; memerlukan petunjuk. | Belum dapat menjelaskan konsep dasar meskipun dengan petunjuk. |

2. Lembar Catatan Observasi

| No. | Nama peserta didik | Pertanyaan/dugaan yang disampaikan | Konsep yang sudah tampak | Dukungan/tindak lanjut |
|---|---|---|---|---|
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |
| 4 | | | | |
| 5 | | | | |
| 6 | | | | |
| 7 | | | | |
| 8 | | | | |

3. Rekap Tindak Lanjut

| Temuan dari observasi | Indikasi kebutuhan | Rencana tindak lanjut |
|---|---|---|
| Beberapa peserta didik belum membedakan abstraksi dan dekomposisi. | Memerlukan contoh pembanding dengan masalah yang sama. | Tampilkan tabel “bagian masalah” dan “informasi penting”, lalu diskusikan. |
| Peserta didik dapat menjelaskan algoritma tetapi belum terbiasa menulis pseudocode. | Memerlukan contoh struktur dan kata kunci. | Berikan template MULAI, BACA, JIKA, TAMPILKAN, SELESAI. |
| Peserta didik dapat menyebut nama fungsi Arduino tetapi belum memahami tujuan. | Memerlukan latihan menghubungkan kebutuhan dengan fungsi. | Gunakan skenario “menerima data / mengirim perintah / menampilkan hasil”. |
| Peserta didik kesulitan menelusuri loop. | Memerlukan penelusuran nilai per iterasi. | Tulis nilai variabel pada setiap putaran dan tandai kondisi berhenti. |
| Partisipasi diskusi terkonsentrasi pada peserta didik yang sama. | Perlu pemerataan kesempatan berbicara. | Gunakan diskusi pasangan, kartu respons, dan giliran menjelaskan. |

## U. GLOSARIUM

| Istilah | Penjelasan ringkas |
|---|---|
| Abstraksi | Memusatkan perhatian pada informasi yang penting dan mengabaikan detail yang tidak diperlukan. |
| Algoritma | Langkah atau aturan yang terstruktur untuk menyelesaikan masalah. |
| Analog I/O | Masukan/keluaran yang berkaitan dengan nilai analog/PWM sesuai fungsi dan kemampuan board. |
| Array | Salah satu bentuk pengaturan data yang disebut dalam pengantar materi. |
| Dekomposisi | Memecah persoalan besar menjadi bagian yang lebih kecil. |
| Decision | Simbol flowchart yang menunjukkan pemilihan jalur berdasarkan kondisi. |
| digitalRead() | Fungsi untuk membaca nilai pin digital berupa HIGH atau LOW. |
| digitalWrite() | Fungsi untuk menuliskan nilai HIGH atau LOW pada pin digital. |
| Flowchart | Diagram yang menggambarkan langkah-langkah algoritma dengan simbol dan arah alur. |
| HIGH / LOW | Keadaan logika pada pin digital yang digunakan dalam materi Arduino. |
| INPUT / OUTPUT | Mode pin untuk menerima masukan atau memberikan keluaran. |
| Inquiry Learning | Model pembelajaran yang mengajak peserta didik membangun pemahaman melalui pertanyaan, dugaan, pencarian informasi, pengujian, dan kesimpulan. |
| Konstanta | Nilai acuan yang ditentukan dan tidak diperlakukan sebagai nilai yang berubah-ubah dalam konteks contoh. |
| Pengenalan pola | Menemukan kesamaan atau keteraturan pada sejumlah contoh atau masalah. |
| Pseudocode | Penulisan langkah algoritma dengan struktur menyerupai kode tetapi tidak terikat pada sintaks bahasa pemrograman tertentu. |
| Serial Monitor | Tampilan yang digunakan untuk melihat data komunikasi serial dari program Arduino. |
| Struktur data | Cara menyimpan dan mengatur data agar mudah digunakan. |
| Variabel | Nama untuk tempat menyimpan nilai yang dapat berubah selama program berjalan. |

| Istilah | Penjelasan ringkas |
|---|---|
| PWM | Pulse Width Modulation, teknik menghasilkan keluaran berbentuk pulsa dengan lebar pulsa yang diatur. |
| while / do-while / for | Struktur perulangan dengan aturan pemeriksaan kondisi dan pengubahan nilai yang berbeda. |

## V. CHECKLIST KETERAMPILAN PESERTA DIDIK

| No | Indikator Keterampilan | Belum Menguasai | Cukup Menguasai | Sangat Menguasai | Catatan |
|---|---|---|---|---|---|
| 1 | Berpikir Komputasional: Mampu menjelaskan dan menerapkan 4 pilar (Dekomposisi, Abstraksi, Pengenalan Pola, Algoritma) pada masalah nyata. | | | | |
| 2 | Representasi Algoritma: Mampu menuliskan algoritma menggunakan bentuk deskriptif, pseudocode, dan flowchart dengan runtut. | | | | |
| 3 | Simbol Flowchart: Mampu mengenali dan menggunakan simbol-simbol flowchart dasar (Terminator, Process, Decision, I/O, dll.) secara tepat. | | | | |
| 4 | Sintaks Dasar Arduino (C/C++): Mampu memahami fungsi komentar, titik koma, kurung kurawal, #define, dan #include. | | | | |

| No | Indikator Keterampilan | Belum Menguasai | Cukup Menguasai | Sangat Menguasai | Catatan |
|---|---|---|---|---|---|
| 5 | Variabel & Konstanta: Mampu mendeklarasikan variabel serta membedakan kelompok konstanta (TRUE/FALSE, HIGH/LOW, INPUT/OUTPUT). | | | | |
| 6 | Komunikasi & Pin I/O: Mampu menggunakan fungsi komunikasi serial (Serial.begin, print, println) serta pin I/O digital/analog. | | | | |
| 7 | Kontrol Alur (Percabangan & Perulangan): Mampu menyusun dan menenusuri alur logika menggunakan if-else, for, while, dan do-while. | | | | |

## W. SUMBER BELAJAR

1) PPT “Berpikir Komputasional dan Algoritma Pemrograman” yang dilampirkan, khususnya pengenalan Algoritma dan Cara Mengatasi masalah berbasis berpikir komputasi.
2) Arduino IDE: Perangkat lunak resmi untuk menulis, mengompilasi, dan mengunggah sketch ke papan mikrokontroler Arduino.
3) Simulator Daring: Wokwi sebagai keperluan simulasi rangkaian dan pengujian logika kode.

## X. PENGESAHAN

**Mengetahui,**

| Guru Mata Pelajaran | Kepala/Wakil Kepala Sekolah |
|---|---|
| Arjun Yuda Firwanda, S.Tr.Kom | Aam Hamzah, S.Pd |